

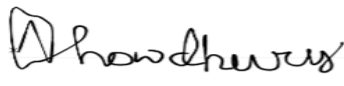
From Complaints to Clarity

A Data-Driven Framework for Consumer Sentiment and Segmentation

Team Members: Shreeshail Shailendra Gupte, Kartik Rajendra Hirijaganer, Nilanjan Chowdhury, Yashvi Hemal Vora

Original Work Statement:

We the undersigned certify that the actual composition of this proposal was done by us and is original work.

	Typed Name	Signature
	Yashvi Hemal Vora	
	Kartik Rajendra Hirijaganer	
	Shreeshail Shailendra Gupte	
	Nilanjan Chowdhury	

INDEX

1.0 Case Study.....	3
1.1 Industry Context and Motivation.....	3
1.2 Business Problem.....	3
1.3 Purpose of Team Project.....	4
1.3.1 Investigative Questions.....	5
1.3.2 Strategic Relevance.....	5
2.0 Implementation.....	6
2.1 Phase 1: Live Streaming System – PySpark + Streamlit.....	6
2.1.2 streamlit_app.py — Live Dashboard.....	8
2.2 Phase 2: Complaint Clustering and Narrative Theme Discovery.....	9
2.2.1 Step 1: Text Preprocessing and Feature Extraction.....	9
2.2.2 Step 2: Dimensionality Reduction Using PCA.....	9
2.2.3 Step 3: Clustering with KMeans.....	10
2.2.4 Step 4: Visualization and Dashboard.....	10
2.2.5 Key Takeaways from Clustering.....	10
2.2.6 Output Files.....	10
2.2.7 Final Thoughts.....	11
2.3 Phase 2: Cloud-Based Complaint Aggregation with Spark SQL on Azure HDInsight.....	11
2.3.1 Step 1: Uploading Data to Azure Blob Storage.....	11
2.3.2 Step 2: Exploratory Aggregations with Spark SQL.....	12
2.3.3 Step 3: Visualization with Matplotlib.....	12
2.3.4 Key Highlights.....	13
2.3.5 Why This Phase Matters.....	13
3.0 Outputs.....	14
3.1 Phase A.....	14
3.2 Phase B.....	15
3.3 Phase C.....	16
4.0 Conclusion.....	18
5.0 Annexures.....	19
5.1 Data Source.....	19
5.2 Data Description.....	19
5.3 Sample Size and Variables.....	20
5.4 Sample Observations.....	20
5.5 Why This Data Is of Interest.....	21

1.0 Case Study

1.1 Industry Context and Motivation

Customer experience has emerged as a critical differentiator for businesses and regulators alike. Every day, thousands of consumers file complaints against financial institutions and service providers through government platforms such as the Consumer Financial Protection Bureau (CFPB). While this data holds valuable insight into customer pain points and service breakdowns, it often remains underutilized due to its volume, variety, and unstructured nature.

Our team recognized an opportunity to unlock this data's potential to support better decision-making—both at the executive level and in day-to-day customer service operations. By leveraging cloud computing, real-time processing, and machine learning, we aimed to turn a static dataset into an evolving system of insights.

1.2 Business Problem

Government agencies and large financial institutions struggle to make timely and meaningful use of customer complaint data. Existing systems often focus only on tracking total complaints by category or time period, without capturing underlying sentiment, channel effectiveness, or changes in consumer tone. This leads to delayed reaction to emerging problems, inefficient complaint resolution strategies, and missed opportunities for systemic improvement.

1.3 Purpose of Team Project

Traditional systems used by regulators and internal compliance teams focus mainly on tracking the volume of complaints, sometimes broken down by product or region. These systems typically rely on monthly or quarterly static reports and do not capture deeper layers such as consumer tone, narrative clustering, submission patterns, or response outcomes. As a result, there is a lag in identifying emerging issues, patterns of dissatisfaction, or gaps in organizational response protocols.

This project explores how a cloud-based big data architecture—combining real-time streaming, large-scale SQL processing, and machine learning—can transform raw complaint data into strategic insights that support timely decision-making and long-term improvement.

We framed our project around the central business question: **“How can consumer complaint data be leveraged to segment customer concerns, detect patterns in complaint channels, and understand sentiment to inform timely and effective responses?”**

To address this question, we structured our project into three interconnected phases:

1. **Real-time Monitoring and Visualization:**

We simulated a live data stream from CFPB API to emulate incoming consumer complaints in real time. This stream was visualized on a dynamic dashboard to help stakeholders quickly identify trends such as spikes in specific issues or states.

2. **Big Data Pipeline and Visualization:**

We fetched updated complaint data from a public API and stored it in Azure HDInsight’s Spark cluster. Using PySpark, we queried this data and visualized complaint trends using visualization libraries like Matplotlib and Seaborn, allowing us to analyze patterns across dimensions such as product categories, complaint channels, and regional distributions.

3. **Unsupervised Segmentation and Sentiment Analysis:**

We further applied unsupervised machine learning (K-Means Clustering) on the structured dataset to segment consumers based on their complaint profiles. Within each cluster, we performed sentiment analysis on the free-text narratives to understand emotional tone and intensity. This allowed us to interpret not just what consumers were complaining about, but how strongly they felt about it, adding qualitative depth to our quantitative clusters.

1.3.1 Investigative Questions

To address these issues, we structure the project around the following **11 business questions**, rooted in the operational and strategic needs of financial service regulators and institutions:

1. What types of financial products receive the highest volume of complaints?
2. Which companies are most frequently complained about, and how do they compare over time or by region?
3. Which states or regions exhibit higher complaint frequencies?
4. What are the most common channels through which consumers submit complaints (web, phone, postal mail, etc.)?
5. How do company responses vary, and which response types are most commonly associated with consumer satisfaction or disputes?
6. What proportion of complaints are disputed by consumers after a response is issued?
7. Are there any noticeable trends or spikes in complaints for specific products, regions, or companies in recent time windows (e.g., real-time or weekly)?
8. How are complaints semantically clustered—what themes or topics are common across different narratives?
9. Can we associate certain clusters or topics with specific companies, product lines, or states?
10. How do submission patterns and sentiment evolve over time or in response to external events (e.g., policy changes, economic shocks)?
11. Can we build a system that continuously ingests, analyzes, and visualizes this data to help regulators and institutions take proactive measures?

1.3.2 Strategic Relevance

Answering these questions will help government bodies:

- Build early-warning systems to detect systemic risk

- Enforce accountability among financial service providers
- Design more responsive regulatory frameworks

Simultaneously, financial institutions can:

- Track complaint resolution KPIs
- Align support strategies with real consumer pain points
- Reduce churn and improve consumer trust

By unifying real-time data processing, SQL-based aggregation, and unsupervised learning into a single project pipeline, we propose a modern solution to an old problem: turning the **voice of the consumer** into a **real-time engine for improvement and compliance**.

2.0 Implementation

2.1 Phase 1: Live Streaming System – PySpark + Streamlit

We have developed a real-time streaming system that simulates complaint data ingestion, performs continuous aggregation using PySpark Structured Streaming, and visualizes results using live Streamlit dashboards. This setup allowed us to monitor complaint trends in near real time.

Our system consists of three key components:

simulate_stream.py — Simulating a Real-Time Stream

To simulate live data, we wrote a Python script that takes the full CFPB complaints dataset and writes it to disk in small chunks every 5 seconds. This mimics a stream of new complaints coming in periodically.

```

import pandas as pd
import time
import os

# Create directory to hold simulated stream
os.makedirs("streaming_data", exist_ok=True)

# Load the dataset
df = pd.read_csv("cfpb_all_complaints.csv")

# Write data in small chunks (100 rows every 5 seconds)
chunk_size = 100
for i in range(0, len(df), chunk_size):
    chunk = df.iloc[i:i + chunk_size]
    chunk.to_csv(f"streaming_data/batch_{i//chunk_size}.csv", index=False)
    print(f"Wrote batch {i//chunk_size}")
    time.sleep(5)

```

This script ensures a continuous stream of data into our pipeline, making it suitable for real-time analysis testing without a Kafka or socket source.

streaming_analysis.py — PySpark Structured Streaming

The second component is our real-time processing engine. We used PySpark Structured Streaming to ingest the CSV files as they arrive in the streaming_data folder. We defined schemas and performed grouped aggregations on product, company, and state fields.

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count

# Initialize Spark
spark = SparkSession.builder.appName("ComplaintStreaming").getOrCreate()

# Read simulated CSV stream
df = spark.readStream.option("header", "true")\
    .schema("product STRING, company STRING, state STRING")\
    .csv("streaming_data")

# Group by product
product_df = df.groupBy("product").agg(count("*").alias("count")).orderBy(col("count").de

# Group by company
company_df = df.groupBy("company").agg(count("*").alias("count")).orderBy(col("count").de

# Group by state
state_df = df.groupBy("state").agg(count("*").alias("count")).orderBy(col("count").desc()

```

This part of the system continuously listens for new batches and updates the aggregated complaint counts accordingly.

2.1.2 streamlit_app.py — Live Dashboard

The final piece is a **Streamlit app** that displays real-time bar charts for each aggregation result. The app refreshes every 5 seconds and shows updated data as it arrives.

This live dashboard gives us instant visibility into complaint trends. We can watch spikes in volume as new data flows in and immediately identify which products or companies are driving the complaints.

2.2 Phase 2: Complaint Clustering and Narrative Theme Discovery

In the final phase of our project, we focused on applying machine learning to uncover hidden themes and insights from unstructured consumer complaint narratives. These narratives contain rich information beyond structured fields, often reflecting consumer frustration, confusion, or dissatisfaction in their own words.

Our goal was to answer this key question:

“What are the underlying themes or clusters of consumer complaints that may not be visible through categorical data alone?”

We built an ML pipeline to process text, reduce dimensionality, and perform clustering—all implemented in PySpark to scale across large datasets.

2.2.1 Step 1: Text Preprocessing and Feature Extraction

We started by selecting relevant fields:

```
df = df.select("complaint_what_happened").na.drop().limit(5000)
```

Then we transformed the text using a simple NLP pipeline:

- Tokenization
- Stopword removal
- Count vectorization

These steps converted free-form complaint text into a structured matrix that machine learning algorithms could work with.

2.2.2 Step 2: Dimensionality Reduction Using PCA

The feature space created by CountVectorizer was extremely sparse and high-dimensional. To make it interpretable and visualization-friendly, we used **Principal Component Analysis (PCA)** to reduce it to 2 components.

This enabled us to visualize each complaint as a point in a 2D space, where proximity indicates semantic similarity.

2.2.3 Step 3: Clustering with KMeans

Using PySpark's MLlib, we applied **KMeans clustering** on the reduced dataset. We selected k=5 clusters to start with, based on qualitative review and balance between granularity and interpretability.

```
from pyspark.ml.clustering import KMeans
```

```
kmeans = KMeans(k=5, seed=1)
```

```
model = kmeans.fit(pca_df)
```

```
result = model.transform(pca_df)
```

Each complaint was now associated with a cluster label, helping us understand the dominant topics in the dataset.

2.2.4 Step 4: Visualization and Dashboard

To make our analysis accessible, we created a **Streamlit dashboard** that displays:

- A **2D scatter plot** of PCA components colored by cluster
- **Cluster-wise summaries** showing the most common complaint patterns
- **Representative narratives** from each cluster

This gave end-users (e.g., regulators or compliance officers) the ability to **explore complaint themes interactively** without needing ML expertise.

2.2.5 Key Takeaways from Clustering

- Some clusters focused heavily on **billing and account errors**, while others highlighted **communication breakdowns** or **unexpected charges**.
- By grouping narratives this way, we were able to reveal latent trends and consumer pain points that structured fields failed to capture.
- This approach allows institutions to not only measure volume, but also understand the **why** behind the complaints.

2.2.6 Output Files

Our pipeline generated the following artifacts:

- **pca_projection.csv**: 2D projection of complaint vectors
- **clustered_output.csv**: Cluster label assigned to each narrative

- **cluster_sentiment_summary.csv**: Common phrases and complaints in each cluster
- **cluster_sentiment_narratives.csv**: Sample complaints from each cluster

These files were used by the Streamlit app to render the visual dashboard.

2.2.7 Final Thoughts

This phase added a **semantic layer of understanding** to our complaint analytics. It went beyond descriptive statistics and empowered decision-makers with **interpretable machine learning insights** from unstructured data. The fact that everything was done using scalable PySpark tools made it enterprise-ready and efficient even on large datasets.

2.3 Phase 2: Cloud-Based Complaint Aggregation with Spark SQL on Azure HDInsight

To demonstrate our ability to handle large-scale data analysis in a **cloud-native environment**, we uploaded the CFPB dataset to **Azure Blob Storage** and connected it with a **Spark cluster** provisioned using **Azure HDInsight**.

This phase was executed entirely through a **Jupyter Notebook hosted on Ambari**, which gave us the flexibility of PySpark while leveraging the computing power of cloud infrastructure.

2.3.1 Step 1: Uploading Data to Azure Blob Storage

We first uploaded the CSV file (cfpb_all_complaints.csv) to a dedicated Azure Blob container. The data was structured, but included some multiline fields, so we configured Spark to read it permissively.

```
csv_path = "wasbs://<container>@<storageaccount>.blob.core.windows.net"
```

```
df = spark.read.option("header", "true")\
```

```
    .option("inferSchema", "true")\
```

```
    .option("multiLine", "true")\
```

```
    .option("mode", "PERMISSIVE")\
```

```
    .csv(csv_path)
```

This setup ensured we could handle malformed rows without breaking the ingestion pipeline.

2.3.2 Step 2: Exploratory Aggregations with Spark SQL

Once the data was successfully loaded into a Spark DataFrame, we began exploring it using grouped aggregations. Our goal was to answer business-centric questions using **large-scale Spark SQL operations**.

We computed complaint counts grouped by:

- **Product**
- **Company**
- **State**
- **Submission Method**
- **Company Response**
- **Dispute Flag (Yes/No)**

Each aggregation was computed using PySpark's `groupBy().agg()` operations and then converted to Pandas for visualization:

```
from pyspark.sql.functions import col, count

product_df = df.groupBy("product").agg(count("*").alias("count"))

submit_df = df.groupBy("submitted_via").agg(count("*").alias("count"))
```

2.3.3 Step 3: Visualization with Matplotlib

Since Spark DataFrames are not directly plottable, we used `.toPandas()` to bring the data into a familiar Python environment and visualized it using `matplotlib`.

We generated the following visual outputs:

- ◆ Bar Chart: Top 10 Companies by Complaint Count

```
company_pd.head(10).plot(kind='bar', x='company', y='count')
```

- ◆ Pie Chart: Complaint Submission Methods

```
submit_pd.plot(kind='pie', y='count', labels=submit_pd['submitted_via'])
```

- ◆ Bar Chart: Complaint Counts by State

```
state_pd.plot(kind='bar', x='state', y='count')
```

- ◆ Horizontal Bar: Company Response Types

```
response_pd.plot(kind='barh', x='company_response', y='count')
```

- ◆ Bar Chart: Disputed vs. Non-Disputed Complaints

```
dispute_pd.plot(kind='bar', x='consumer_disputed', y='count')
```

These charts helped us quickly summarize which companies and regions have the highest complaint activity, how consumers are interacting with the system, and how companies are responding.

2.3.4 Key Highlights

- We successfully connected Azure Blob Storage to Spark on HDInsight using [wasbs://](#), demonstrating cloud-native data ingestion.
- All data operations were performed using PySpark, showcasing our ability to scale processing with large complaint datasets.
- Visualizations were generated in Jupyter using pandas + matplotlib, offering a flexible hybrid environment (big data backend + Python frontend).
- We explored both **structured trends** (like company volume) and **behavioral trends** (like dispute rates and channel preference).

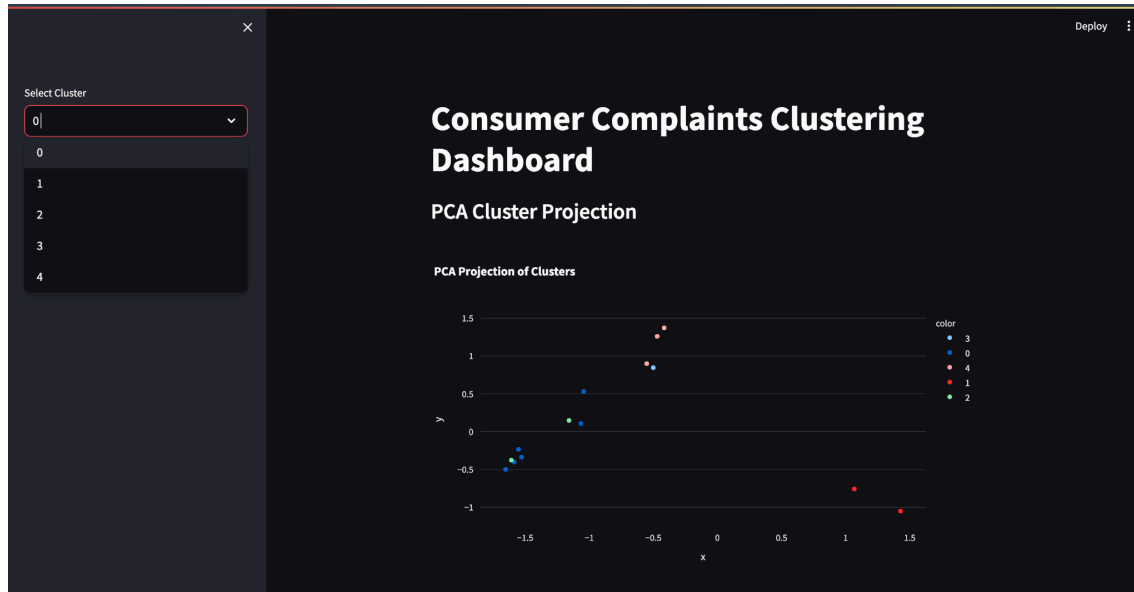
2.3.5 Why This Phase Matters

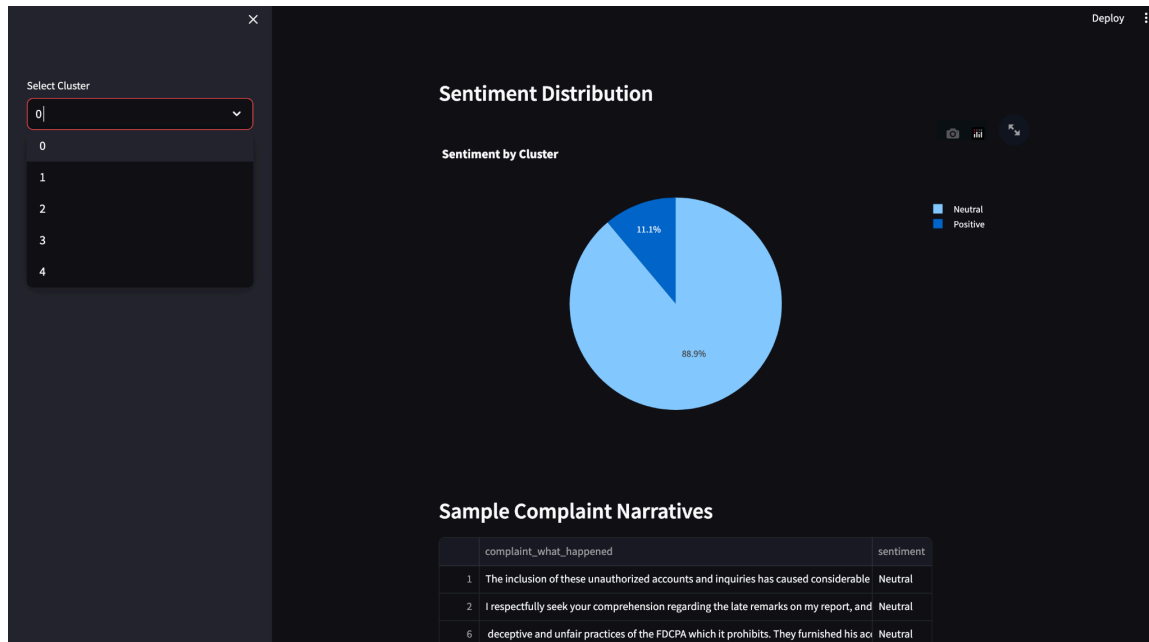
This notebook simulates a **real-world analytics pipeline** for an enterprise data team. It mirrors how a government or financial institution might use cloud resources to conduct quarterly audits or drill into customer feedback trends at scale.

It also complements our real-time streaming system—together, they form a complete **batch + streaming architecture** capable of answering both static and dynamic business questions.

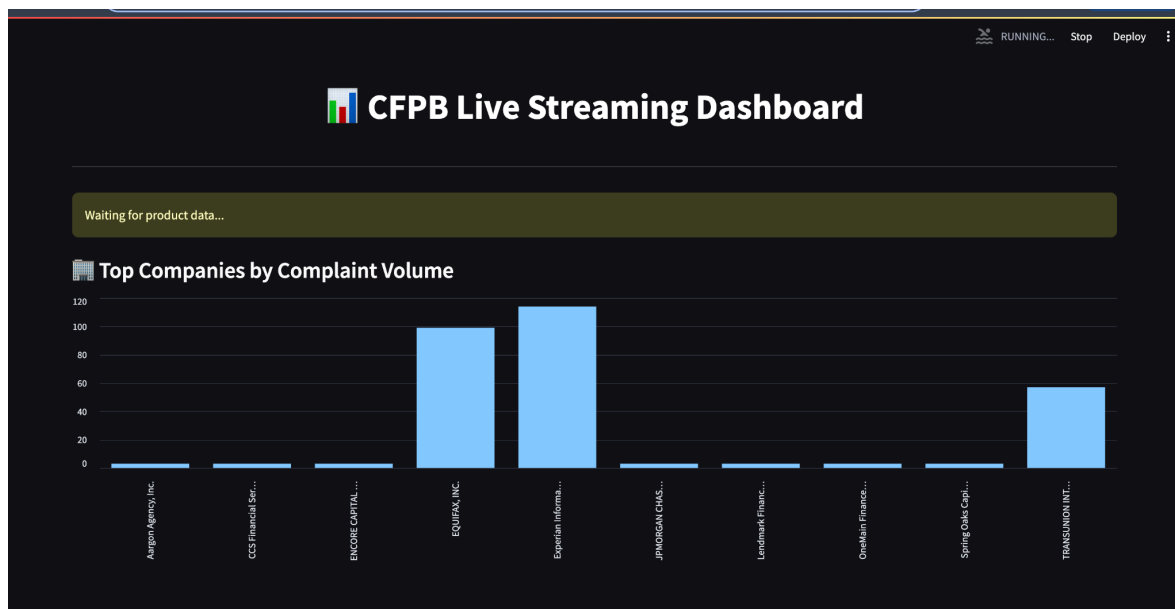
3.0 Outputs

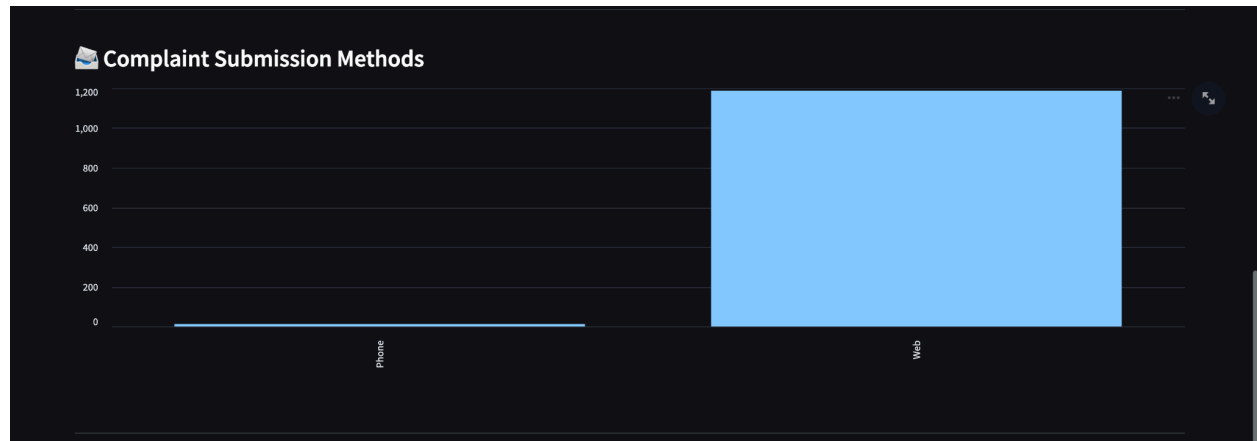
3.1 Phase A



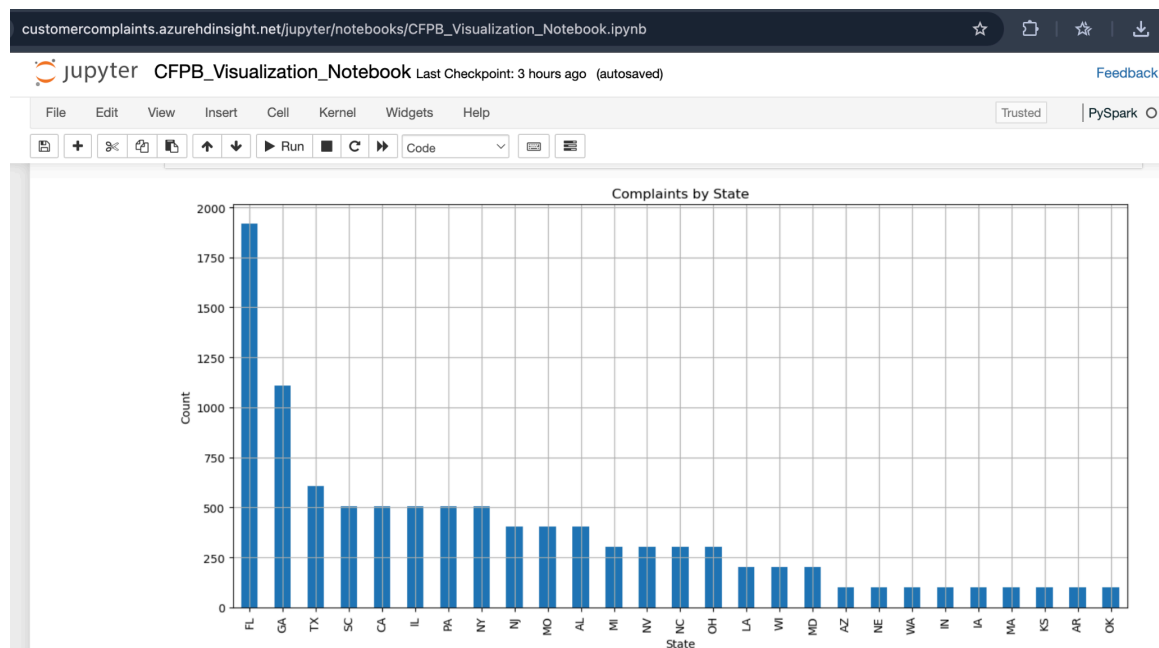


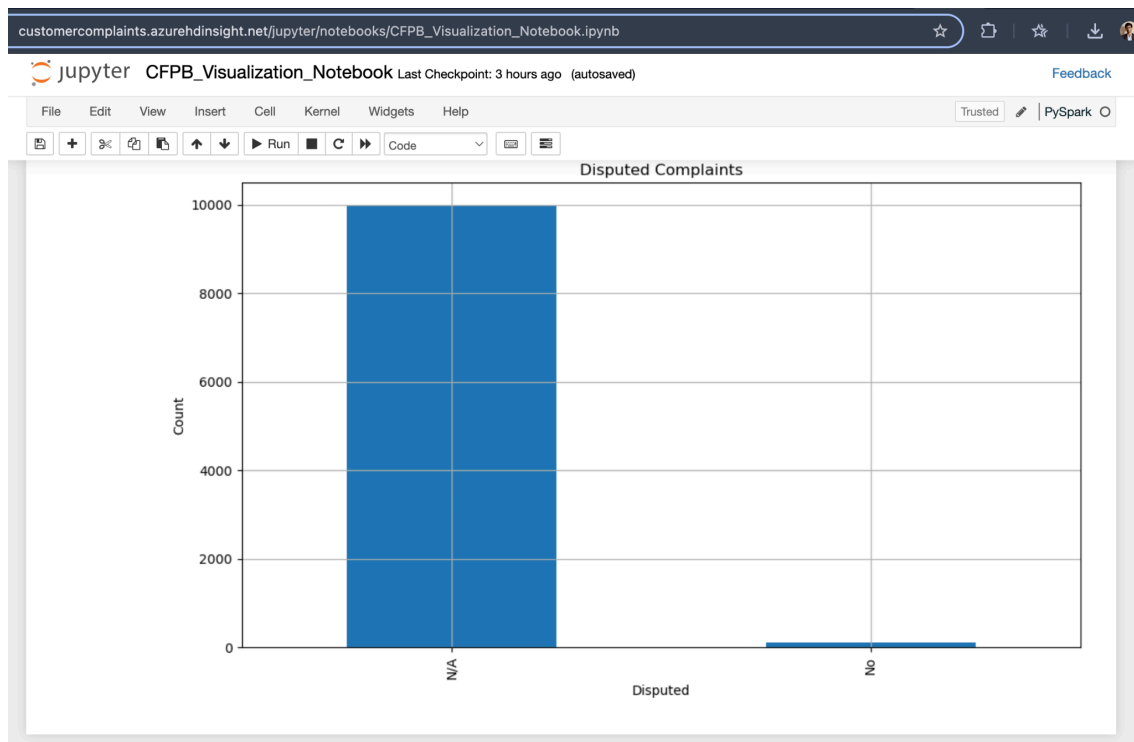
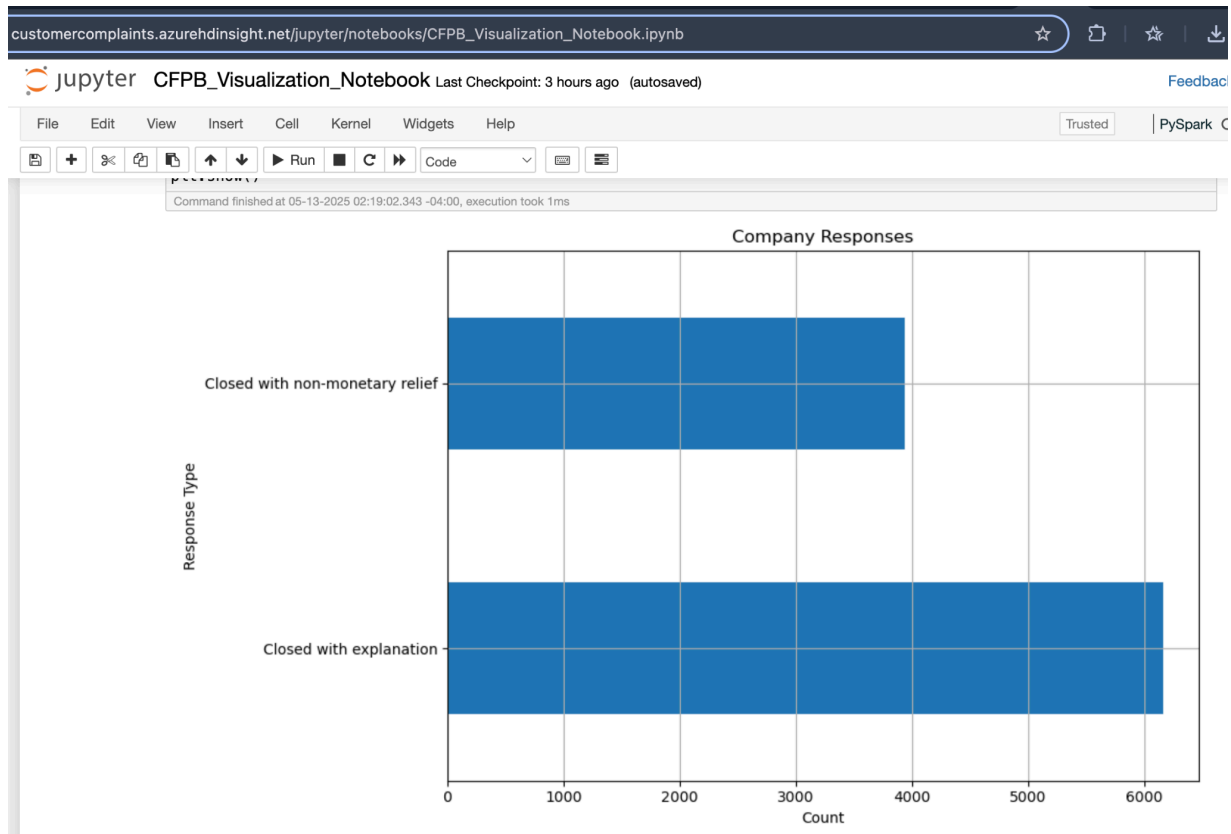
3.2 Phase B

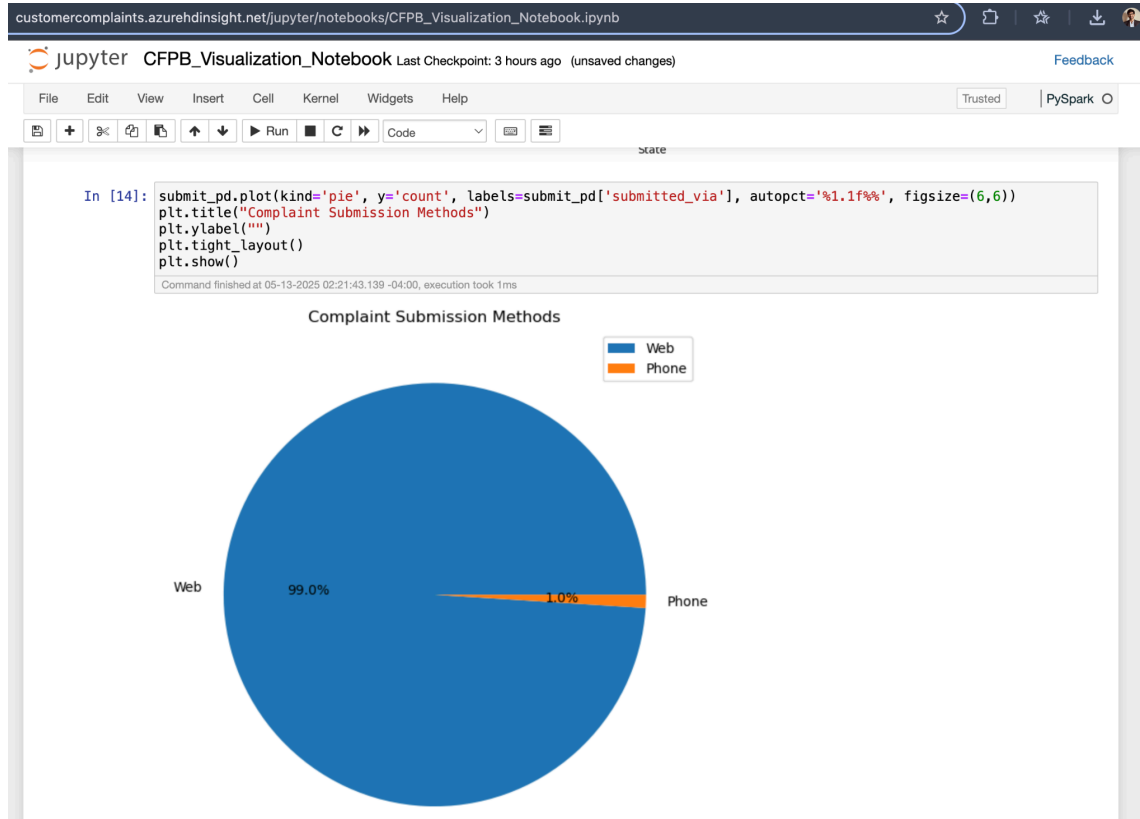




3.3 Phase C







4.0 Conclusion

Our project successfully demonstrated how advanced cloud computing and big data technologies can be harnessed to transform unstructured consumer complaint data into meaningful, actionable insights. By combining real-time streaming with PySpark, scalable cloud-based SQL processing on Azure HDInsight, and machine learning techniques such as K-Means clustering and sentiment analysis, we built an end-to-end analytics pipeline that addresses both operational and strategic needs. The insights derived from our clustering models and narrative analysis not only revealed hidden consumer pain points but also enhanced interpretability for stakeholders through dynamic dashboards. Ultimately, our work presents a scalable, enterprise-ready solution that can empower financial institutions and regulators to proactively address customer grievances, monitor emerging trends, and drive data-informed policy improvements. This case study validates the power of integrating structured and unstructured data to support continuous consumer intelligence in a fast-evolving regulatory landscape.

5.0 Annexures

5.1 Data Source

The dataset used in our project was obtained from the **Consumer Financial Protection Bureau (CFPB)**, a U.S. government agency that maintains a publicly accessible database of consumer complaints against financial institutions. The original and most updated version of the data is available through the CFPB's official API:

<https://www.consumerfinance.gov/data-research/consumer-complaints/>

5.2 Data Description

The dataset captures complaints submitted by consumers regarding financial products and services. Each row represents one consumer complaint and includes both structured attributes and narrative text.

Variable Name	Description	Type
date_received	Date complaint was received	Date
product	Main product involved (e.g., Credit card, Mortgage)	Categorical
sub_product	Specific product category (e.g., Conventional mortgage)	Categorical
issue	Type of problem described	Categorical
sub_issue	More detailed issue classification	Categorical
consumer_complaint_narrative	Free-text description of the complaint	Text
company	Company the complaint was submitted against	Categorical
state	State where consumer lives	Categorical
zip_code	ZIP code of consumer	Categorical

submitted_via	Channel through which complaint was submitted (e.g., Web, Phone)	Categorical
date_sent_to_company	When the complaint was forwarded to the company	Date
company_response_to_consumer	How the company responded	Categorical
timely_response	Whether the company responded in a timely manner	Boolean
consumer_consent_provided	Whether consumer consented to publishing narrative	Categorical
has_narrative	Boolean field indicating if narrative text is available	Boolean

5.3 Sample Size and Variables

1. Sample Size (n): ~2.8 million rows (varies slightly with updates)
2. Number of Variables (k): 18 structured fields + 1 narrative field (text)

5.4 Sample Observations

date_received	product	issue	company	state	submitted_via	has_narrative	consumer_complaint_narrative
2022-06-10	Credit card	Billing disputes	ABC Bank	CA	Web	True	"I was charged late fees..."
2022-06-12	Mortgage	Loan servicing, payments	XYZ Mortgage	TX	Phone	False	
2022-06-14	Debt collection	Communication tactics	Debt Inc.	NY	Email	True	"They called me every day..."

2022-06-15	Check ing accou nt	Account opening/cl osing	Bank of Test	FL	Web	True	"The account was closed without..."
------------	-----------------------------	--------------------------------	--------------------	----	-----	------	--

5.5 Why This Data Is of Interest

This dataset is highly relevant for multiple reasons:

1. **Policy impact:** Provides regulators with insight into recurring consumer issues.
2. **Business decision-making:** Allows companies to proactively address service issues.
3. **Public value:** Offers transparency into how companies respond to consumer grievances.
4. **Rich analytical potential:** Combines structured and unstructured data, ideal for machine learning, sentiment analysis, and real-time dashboards.